

Resumen técnico de los modelos

nba_predictor · cascada $M1 \rightarrow M2 \rightarrow M3 \rightarrow M4$

Para cada modelo: qué parámetros usa, la parte matemática, la parte de cómputo y por qué está hecho así.

0. Insumos que alimentan la cascada

Antes de los modelos, tres cantidades derivadas de los datos crudos alimentan la cadena. Se resumen aquí porque su matemática reaparece en M1-M4.

PIE (impacto del jugador)

Estadística oficial de la NBA. Mide la porción del total de acciones positivas de un partido que aportó el jugador:

```
PIE = (PTS+FGM+FTM-FGA-FTA+DREB+0.5*OREB+AST+STL+0.5*BLK-PF-TOV)
      / (mismos totales sumados de TODO el partido)
```

squad_strength (fuerza del plantel)

Promedio del PIE de los jugadores ponderado por minutos, estandarizado por temporada:

```
squad_strength = z_temporada( sum_i (min_i / sum_min) * PIE_i )
```

Proyección de jugador (curva de edad)

Para no usar datos del año a predecir, el PIE de cada jugador para T se estima desde su historia con recencia + regresión a la media + edad:

```
PIE_proj = base * mult(edad_T)/mult(edad_ult) (curva estandar)
base = ( n*media_reciente + k*prior ) / (n + k)
```

La fuerza proyectada del equipo (squad_strength_proj) repite la fórmula de squad_strength pero con PIE y minutos proyectados. Correlaciona 0.61 con las victorias reales sin mirar el año en curso.

M1 — Modelo de victorias (regresión Ridge)

Parámetros que usa

Cinco variables por equipo-temporada, elegidas por backtest+ablacion: **continuity** (continuidad de plantilla), **prior_net_rating** (diferencial del año previo), **avg_age_core** (edad del núcleo), **squad_strength_proj** (fuerza proyectada) y **best_pie_proj** (la estrella). El entrenador se probó pero se descartó: empeoraba el backtest. Objetivo a predecir: victorias reales de la temporada.

Parte matemática

Es un modelo lineal regularizado. Primero se estandariza cada variable (media 0, desviación 1) usando solo el conjunto de entrenamiento:

```
~z = (x - media_train) / desv_train
```

Luego Ridge ajusta los coeficientes minimizando el error cuadrático MÁS una penalización L2 que castiga coeficientes grandes (alfa = 1):

```
~beta = argmin sum (y - X*beta)^2 + alfa * ||beta||^2
```

que tiene solución cerrada $\beta = (X^T X + \alpha I)^{-1} X^T y$. La predicción es $y = X \beta$, recortada al rango válido [0, 82] victorias.

Parte de cómputo

Se implementa con StandardScaler + Ridge de scikit-learn (solución algebraica exacta, sin iteración). La validación es walk-forward: para predecir la temporada T se reentrena con TODAS las temporadas anteriores y se predice T; se avanza en el tiempo. Coste trivial (matrices 5x5).

Por qué lo hace así

Con pocas filas (30-150) y 5 variables, una regresión normal (OLS) podría sobreajustar el ruido; la penalización L2 encoge los coeficientes y da estabilidad (se probó GradientBoosting y RandomForest: peores). La estandarización hace la penalización justa entre variables y los pesos comparables (continuity pesa +4.4, prior_net_rating +2.7). El walk-forward evita fuga temporal. Se recorta a [0,82] porque no existen récords fuera de ese rango.

Resultado: MAE 7.02 victorias (IC 90% [6.3, 7.8]), mejor que la persistencia (8.89) en las 5 temporadas. Sobre el calendario, 6.96 y ~63% de acierto por partido.

M2 — Seeding por conferencia (ordenamiento)

Parámetros que usa

Las victorias proyectadas por M1 (wins_pred) y la conferencia de cada equipo (Este/Oeste, tabla fija).

Parte matemática

No hay entrenamiento: es un ordenamiento. Dentro de cada conferencia se asigna el seed por el rango de las victorias proyectadas (mayor a menor):

```
~seed = rank_desc( wins_pred ) dentro de la conferencia
```

y el tramo sale del seed: 1-6 Playoffs, 7-10 Play-in, 11-15 Lotería.

Parte de cómputo

Un groupby por (temporada, conferencia) y una función de rango. Instantáneo. Se valida comparando el seed predicho con el real (correlación de Spearman) y el acierto de tramo.

Por qué lo hace así

La NBA siembra a los equipos por su récord dentro de la conferencia; ordenar las victorias proyectadas es el análogo directo de esa regla. No necesita un modelo aparte: M2 traduce el número continuo de M1 a la estructura discreta (posiciones y tramos) que usan los playoffs. Hereda el error de M1, así que se valida en conjunto.

Resultado: correlación de seed 0.68; ~72% de los clasificados a playoffs identificados.

M3 — Simulación de playoffs (Monte Carlo)

Parámetros que usa

Los seeds de M2 (bracket) y la calidad de cada equipo $q = \text{wins_pred}/82$, recortada a $[0.05, 0.95]$. Una ventaja de local $HCA = 0.06$.

Parte matemática

Probabilidad de que A gane UN partido a B, dadas sus tasas de victoria (fórmula log5 de Bill James):

$$\sim p = (q_A - q_A * q_B) / (q_A + q_B - 2 * q_A * q_B)$$

Al que juega de local se le suma la ventaja: $p_{\text{local}} = p + HCA$ (recortada). El patrón de sedes es 2-2-1-1-1 para el mejor seed.

Probabilidad de ganar la SERIE (al mejor de 7): se calcula exacta por programación dinámica sobre (juego i , victorias A, victorias B):

$$\sim P(i, w_A, w_B) = p * P(i+1, w_A+1, w_B) + (1-p) * P(i+1, w_A, w_B+1)$$

base: $w_A=4 \rightarrow 1$; $w_B=4 \rightarrow 0$

El campeón surge simulando el bracket completo muchas veces; la probabilidad de cada evento es su frecuencia:

$$\sim P(\text{evento}) \approx (\text{veces que ocurre}) / N_{\text{sims}}$$

Parte de cómputo

La probabilidad de serie usa recursión con memoización (`lru_cache`): exacta y barata. Sobre eso se corre Monte Carlo con $N = 20.000$ simulaciones por temporada; en cada una se muestrean los ganadores de serie con un generador aleatorio y se avanza el bracket (R1, semis, final de conferencia, Finales). Tarda ~14 s. El error de muestreo es $\sim \sqrt{p(1-p)/N} \approx 0.3\%$.

Por qué lo hace así

Los playoffs son un árbol con azar: quién enfrenta a quién en la ronda siguiente depende de resultados anteriores, así que una fórmula cerrada para la probabilidad de campeón es enredada. Monte Carlo propaga esa incertidumbre de forma natural. Se usa log5 porque es la manera principista de obtener un enfrentamiento directo a partir de dos tasas de victoria; y se calcula la serie exacta por DP para que la única variación venga del bracket, no del cálculo de cada serie.

Resultado: Brier 0.059 sobre el campeón; da probabilidades realistas y bajas al campeón preseason (lo esperable).

M4 — Probabilidad de campeón (calibración)

Parámetros que usa

La probabilidad de título de M3 (p_{champion}) y la proximidad de construcción a campeón proyectada (champ_sim_proj). Un peso w que se elige.

Parte matemática

M4 reajusta las probabilidades de M3 empujando a los equipos que se arman como los campeones. Con z = z-score de la proximidad por temporada:

```
~score = p_M3 * exp( w * z_proximidad )  
p_M4 = score / suma(score) (normaliza por temporada)
```

El peso w se elige por el que minimiza la log-loss del campeón en backtest:

```
~LL = - promedio( y*log(p) + (1-y)*log(1-p) )
```

Parte de cómputo

Se prueba w en una rejilla $\{0, 0.25, 0.5, 0.75, 1.0\}$, se calcula la log-loss de cada uno y se queda el mejor. Operación ligera.

Por qué lo hace así

El reajuste multiplicativo con $\exp$ mantiene las probabilidades positivas y que sumen 1; es una reponderación de estilo bayesiano por un 'prior' de construcción. Elegir w por backtest es selección de modelo. Aquí el mejor resultó $w = 0$: la proximidad de construcción NO mejora la predicción del campeón (la empeora), así que se descarta. Es la misma disciplina que con el entrenador: una señal entra solo si el backtest la respalda. Por eso, hoy, $M4 = M3$.

Resultado: $w = 0$ (log-loss 0.261). La mejor estimación de campeón es la propia simulación de M3.

Panorama de la cascada

M1 convierte talento proyectado + continuidad + récord previo + entrenador en victorias (Ridge). **M2** ordena esas victorias en seeds por conferencia. **M3** simula el bracket miles de veces (log5 + DP + Monte Carlo) para sacar probabilidades por ronda y de título. **M4** intenta calibrar el título con la construcción de plantel, y el backtest dice que no aporta, así que se mantiene M3.

Dos principios recorren todo: **nada de fuga temporal** (para predecir T solo se usa información anterior a T) y **cada variable o señal entra solo si mejora el backtest**. Por eso el sistema es honesto y se puede seguir extendiendo con confianza.